`Machine` might have properties such as `serialNumber`, `operationalStatus`, and `lastMaintenanceDate`.

- *Relationships* define how different concepts are interrelated, mapping the complex dependencies that exist in the physical world. For example, a relationship might state that a `Product` is `processedBy` a `Machine`. These connections allow the digital twin to understand the contextual relationships between data.
- *Rules and constraints* govern the behaviour and valid states of entities and relationships, allowing for automated reasoning and validation. For example a rule might state that a `Product` cannot be `processedBy` a `Machine` whose `operationalStatus` is `offline`.

5.3.2 Defining ontologies for digital twins

To define an ontology, we use a formal language that is both human-readable and machine-interpretable. In the context of digital twins, a leading example of such a language is the Digital Twin Definition Language (DTDL) used by platforms like Microsoft's Azure Digital Twin service. DTDL provides a way to define the interfaces of a digital twin, specifying the models and relationships that a twin instance can have. It uses a JSON-LD syntax, which makes it easy to read and write. The language is structured to precisely capture the concepts (known as interfaces), properties (telemetry, properties), and relationships of a real-world entity.

NOTE

JSON-LD (JSON for linking data https://json-ld.org/) is a method of encoding linked data using JSON designed to add semantic meaning to regular JSON by linking it to shared vocabularies and ontologies on the web.

For example, an interface for a room in a building would define its properties like whether it is occupied, and its temperature as shown in a minimal example in listing 5.2.